

Large Scale Police Crime Data Analytics with Spark SQL

Neha Vadapally



<i>Analysing Data with Spark SQL</i>	<i>2</i>
1. Introduction of dataset	3
1.1 Data Source, Scope and Coverage.....	3
1.2 Dataset Structure, Key Variables and Assumptions	4
2. Data Analysis.....	4
2.1 Distributed Data Ingestion and Preparation Using Spark SQL	5
2.2 Data Cleaning, Validation and Feature Engineering	5
2.3 Temporal Analysis of Crime Trends Across Forces.....	7
2.4 Cross-Force Comparative Analysis of Crime Volume	9
2.5 Crime-Type Composition and Distribution Analysis.....	11
2.6 Outcome Resolution and Spatial (LSOA-Level) Analysis	12
3. Discussion	14
3.1 Interpretation of Key Analytical Results.....	15
3.2 Practical Usefulness of Insights and Methodological Limitations.....	15
3.3 Critical Reflection on the Use of Spark SQL for Large-Scale Analytics	15
3.4 Conclusions and Directions for Future Work	15
<i>References and Bibliography.....</i>	<i>15</i>
<i>Appendix.....</i>	<i>16</i>

Analysing Data with Spark SQL

1. Introduction of dataset

1.1 Data Source, Scope and Coverage

The aim of this study is to investigate police crime statistics on the street level and the data for this investigation was retrieved from the Police.uk open data portal, (Police.uk, 2026). The data was retrieved from a series of monthly data files from various UK police forces and it includes information such as the nature of the crime, the outcome of the investigation, the approximate geographical coordinates and the name of the lower-layer super output area. The data includes crime reports of 4,059,565 from 25 consecutive months between November 2023 and November 2025, based on programmed analysis of the directory structure and data ingestion.

```
from pathlib import Path

ROOT = Path("/kaggle/input/t1-police-data")
if not ROOT.exists():
    raise FileNotFoundError(f"Root path not found: {ROOT}")

months = sorted([p for p in ROOT.iterdir() if p.is_dir()])
print(f"Months detected: {len(months)}")
print([p.name for p in months[:6]])

# Sample files in the first two month folders
for p in months[:2]:
    sample = sorted([f.name for f in p.glob("*.csv")])[:6]
    print(f"\nMonth folder: {p.name}")
    for s in sample:
        print(" ", s)

Months detected: 25
['2023-11', '2023-12', '2024-01', '2024-02', '2024-03', '2024-04']

Month folder: 2023-11
2023-11-essex-street.csv
2023-11-leicestershire-street.csv
2023-11-metropolitan-street.csv
2023-11-norfolk-street.csv
2023-11-northern-ireland-street.csv
2023-11-west-midlands-street.csv

Month folder: 2023-12
2023-12-essex-street.csv
2023-12-leicestershire-street.csv
2023-12-metropolitan-street.csv
2023-12-norfolk-street.csv
2023-12-northern-ireland-street.csv
2023-12-west-midlands-street.csv
```

Figure 1: Structural overview of the raw dataset.

The police forces included in this analysis are the Metropolitan Police Service, West Midlands Police, Essex Police, Leicestershire Police, Norfolk Constabulary and Police Service of Northern Ireland. The inclusion of numerous police forces exceeds the basic coursework requirement and offers ample opportunities to make cross-force comparisons. The spatial coordinates provided by Police.uk are anonymous and cannot be used to make accurate

spatial analysis, which is a well-known problem with UK open crime data (Tompson et al., 2015).

1.2 Dataset Structure, Key Variables and Assumptions

Each record corresponds to a single recorded incident and it has associated temporal, categorical and spatial attributes. The primary analytical columns are crime_type, outcome, falls_within (police force) and derived columns for time. Spatial attributes are also included, but only as an approximation using latitude and longitude. Although it has been recognized that recording practices may vary across different countries, it has been believed that column semantics are consistent across armies.

crime_id	outcome	month_str	reported_by	falls_within
crime_type		lon	lat	
94cce8effdc939bce021aacd07b5d33a8389265fde201c18e651fbf849f78b5	2025-07-01 00:00:00	Metropolitan Police Service	Metropolitan P	olice Service Violence and sexual offences Unable to prosecute suspect -0.250117 50.837908
4ba6d869dcd0556337c067118d34af3597d38adfa11fb8f847e09ab359d91f08	2025-07-01 00:00:00	Metropolitan Police Service	Metropolitan P	olice Service Public order Under investigation -3.540359 54.635844
6e36980c64aafe2daa7f34435ec09dfcea296b90010fb2480d58842a814fb930	2025-07-01 00:00:00	Metropolitan Police Service	Metropolitan P	olice Service Violence and sexual offences Under investigation -0.65746 50.78727
e5ca230a94af8c4da9530f6287e4aa7f912177d9f3439e0c56a4a978a6c10d8e	2025-07-01 00:00:00	Metropolitan Police Service	Metropolitan P	olice Service Violence and sexual offences Under investigation -1.251798 53.128747
e4420265bcc386307fc1bc74b47524bb14450484b7eddee4bc0e7534e3e690a8	2025-07-01 00:00:00	Metropolitan Police Service	Metropolitan P	olice Service Violence and sexual offences Under investigation -1.263473 53.121601

only showing top 5 rows

Figure 2: Summary statistics obtained from Figure 17.

Type casting and normalization are employed to ensure data consistency across monthly data sets. This is a fundamental requirement for a distributed analytics approach described by Zaharia et al. (2016).

2. Data Analysis

2.1 Distributed Data Ingestion and Preparation Using Spark SQL

All monthly CSV files are ingested using Spark's distributed CSV reader, enabling efficient processing of over four million records (see Figure 15). Schema inference is used to accommodate heterogeneous raw files, while column headers are standardised for analytical clarity. Longitude and latitude fields are explicitly cast to numeric types to prevent downstream aggregation errors (See Figures 3 and 16).

```
YEAR_RE = r".*/(\d{4})-(\d{2})-[a-z-]+-street\.csv$"
MON_RE = r".*/(\d{4})-(\d{2})-[a-z-]+-street\.csv$"

df_prep = (
    df_norm
    .withColumn("year", F.regexp_extract("file_path", YEAR_RE, 1))
    .withColumn("month_num", F.regexp_extract("file_path", MON_RE, 1))
    .withColumn("year_month", F.concat_ws("-", "year", "month_num"))
    .withColumn("force", F.col("falls_within"))
)

df_prep.createOrReplaceTempView("crimes")

df_prep.select(
    "file_path", "year", "month_num",
    "year_month", "force"
).show(6, truncate=False)
```

Figure 3: Code snippet to extract values from file name

Temporal attributes including year, month and year-month are derived directly from filenames using regular expressions (Tukey, 1977), avoiding reliance on inconsistent timestamp formats. Police force attribution is derived dynamically from the falls_within field rather than hard-coded mappings, improving robustness and generalisability of the pipeline.

2.2 Data Cleaning, Validation and Feature Engineering

The extent of variance in data completeness is indicated by a global null-value profile (See Figures 4 and 32). As the Context field is entirely absent, it is removed because of analytical redundancy (Little and Rubin, 2019). The data for the outcome of investigation shows 22.22 percent missingness, while 17.35 percent of the data lacks a criminal identification. On the contrary, the absence of missing data in crime category, force and time variables is conducive to valid aggregation.

```

null_profile_df = spark.createDataFrame([
    (c,
     df_prep.filter(F.col(c).isNull()).count(),
     round(100 * df_prep.filter(F.col(c).isNull()).count() / df_prep.count(), 2)
    )
    for c in df_prep.columns
], ["column_name", "null_count", "null_pct"])

null_profile_df.orderBy(F.desc("null_pct")).show(truncate=False)

```

```

[Stage 154:>                                     (0 + 4) / 4]
+-----+-----+-----+
|column_name|null_count|null_pct|
+-----+-----+-----+
|Context    |4059565   |100.0   |
|outcome    |901905    |22.22   |
|crime_id   |704479    |17.35   |
|lsoa_code  |305787    |7.53    |
|lsoa_name  |305787    |7.53    |
|lon        |19701     |0.49    |
|lat        |19701     |0.49    |
|file_path  |0         |0.0     |
|month_str  |0         |0.0     |
|year       |0         |0.0     |
|crime_type |0         |0.0     |
|month_num  |0         |0.0     |
|reported_by|0         |0.0     |
|year_month |0         |0.0     |
|location   |0         |0.0     |
|force      |0         |0.0     |
|falls_within|0        |0.0     |
+-----+-----+-----+

```

Figure 4: Missing data patterns motivating pre-processing decisions

Temporal consistency is verified through integrity checks, while validation confirms missing identifiers relate to anti-social behaviour (ASB). Outcomes are coded as “Outcome not recorded,” following public-sector data-quality standards (Batini et al., 2009) (Figures 5,6 and 19).

<pre>print("Outcome categories after cleaning:") spark.sql(""" SELECT outcome_clean, COUNT(*) AS records FROM crimes_final GROUP BY outcome_clean ORDER BY records DESC """).show(truncate=False)</pre>	<pre>outcome_clean records +-----+-----+ Investigation complete; no suspect identified 1503467 Unable to prosecute suspect 1005131 Outcome not recorded 901905 Under investigation 163479 Status update unavailable 145909 Court result unavailable 109316 Awaiting court outcome 76359 Local resolution 74074 Action to be taken by another organisation 38692 Offender given a caution 15776 Formal action is not in the public interest 10606 Further action is not in the public interest 4731 Offender given penalty notice 4608 Further investigation is not in the public interest 4403 Suspect charged as part of another case 1077 Offender given a drugs possession warning 32 +-----+-----+</pre>

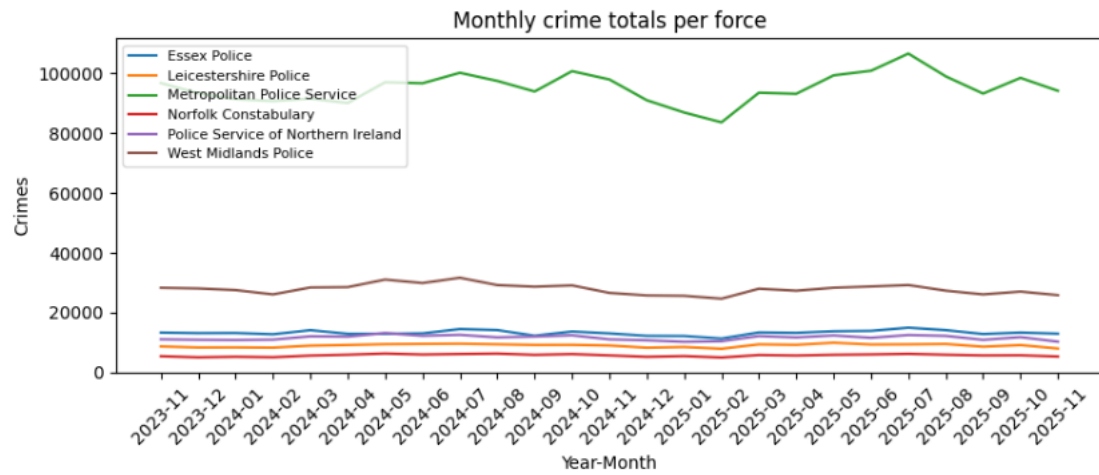
Figure 5: Handling missing outcome values

<pre>print("ASB flag distribution:") spark.sql(""" SELECT is_asb, COUNT(*) AS records FROM crimes_final GROUP BY is_asb """).show(truncate=False)</pre>	<pre>ASB flag distribution: [Stage 167:=====] +-----+-----+ is_asb records +-----+-----+ 1 704479 0 3355086 +-----+-----+</pre>
---	--

Figure 6: Missing crime_id indicating ASB (7,04,479)

2.3 Temporal Analysis of Crime Trends Across Forces

Monthly and yearly aggregations are computed using Spark SQL group-by operations. Figures 7 and 8 illustrates broadly stable monthly crime volumes across all forces, indicating that short-term fluctuations are modest relative to long-term structural differences.



<Figure size 800x400 with 0 Axes>

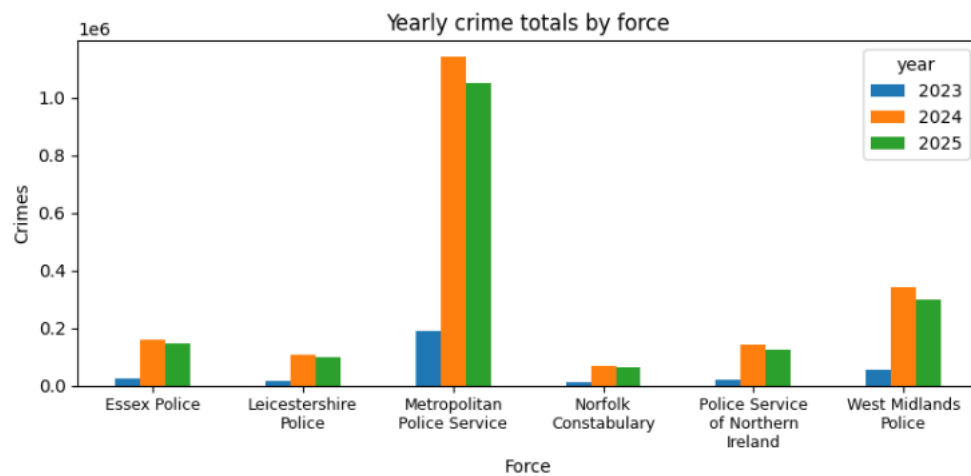


Figure 7: Monthly and yearly crime trends across police forces

Figures 7 and 9 illustrate the continuous scale, showing the Metropolitan Police Service recorded 1.19 million crimes in 2023, increasing to 1.45 million in 2024, whereas the Norfolk force recorded less than 70,000.

+-----+-----+-----+-----+			
force	year	month_num	month_total
+-----+-----+-----+-----+			
Essex Police	2023	11	13292
Essex Police	2023	12	13110
Essex Police	2024	01	13148
Essex Police	2024	02	12722
Essex Police	2024	03	14092
Essex Police	2024	04	12875
Essex Police	2024	05	12872
Essex Police	2024	06	13020
Essex Police	2024	07	14471
Essex Police	2024	08	14125
+-----+-----+-----+-----+			
only showing top 10 rows			

Figure 8: Monthly totals by force

+-----+-----+-----+		
year	force	total_crimes
+-----+-----+-----+		
2023	Metropolitan Police Service	190199
2023	West Midlands Police	56327
2023	Essex Police	26402
2023	Police Service of Northern Ireland	21908
2023	Leicestershire Police	16925
2023	Norfolk Constabulary	10380
2024	Metropolitan Police Service	1139329
2024	West Midlands Police	342166
2024	Essex Police	158403
2024	Police Service of Northern Ireland	141256
+-----+-----+-----+		
only showing top 10 rows		

Figure 9: Yearly totals by force

2.4 Cross-Force Comparative Analysis of Crime Volume

Figures 10, 20 and 21 illustrate the proportional share, whereas Figure 10 confirms the consistent position of Metropolitan police Service at first, followed by West Midlands Police force's in second place.

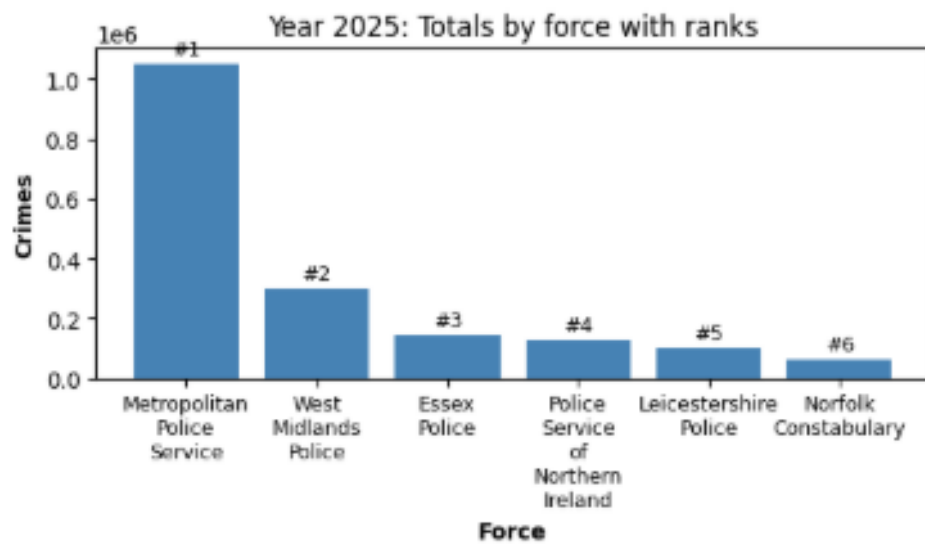
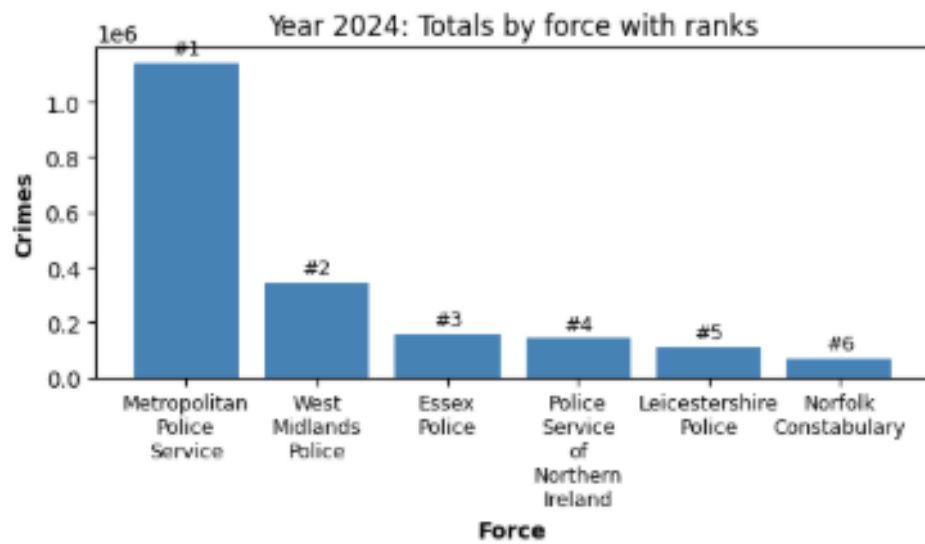
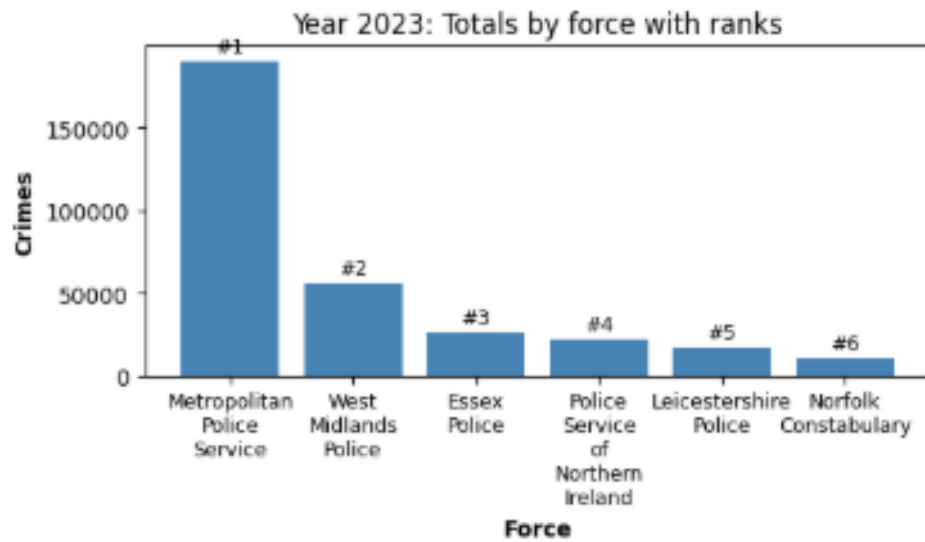


Figure 10: Yearly crime totals with rankings

year	force	total_crimes	pct_share
2023	Metropolitan Police Service	190199	59.04
2023	West Midlands Police	56327	17.49
2023	Essex Police	26402	8.20
2023	Police Service of Northern Ireland	21908	6.80
2023	Leicestershire Police	16925	5.25
2023	Norfolk Constabulary	10380	3.22
2024	Metropolitan Police Service	1139329	58.19
2024	West Midlands Police	342166	17.47
2024	Essex Police	158403	8.09
2024	Police Service of Northern Ireland	141256	7.21
2024	Leicestershire Police	107902	5.51
2024	Norfolk Constabulary	69028	3.53
2025	Metropolitan Police Service	1049455	58.98
2025	West Midlands Police	297884	16.74
2025	Essex Police	145611	8.18
2025	Police Service of Northern Ireland	125737	7.07
2025	Leicestershire Police	98432	5.53
2025	Norfolk Constabulary	62221	3.50

Figure 11: Cross-force ranking and share comparison

In Figure 11, the Metropolitan Police Service recorded 59% of all crimes, thus illustrating inherent scale differences, which create difficulties in the performance analysis of various police forces of different scales.

2.5 Crime-Type Composition and Distribution Analysis

Crime type distributions are derived from force-level aggregate searches. Figure 12 illustrates that violent and sexual crimes are the most predominant, followed by anti-social conduct and public order crimes, which are consistent with the cross-force crime type distributions identified by Chainey and Ratcliffe (2013). The consistency of these trends is a clear indication that crime profiles are becoming more similar, despite differences in the quantity of crime.

force	crime_type	ct
Essex Police	Violence and sexual offences	131875
Essex Police	Anti-social behaviour	29876
Essex Police	Shoplifting	27972
Leicestershire Police	Violence and sexual offences	80636
Leicestershire Police	Anti-social behaviour	26039
Leicestershire Police	Public order	20763
Metropolitan Police Service	Violence and sexual offences	546701
Metropolitan Police Service	Anti-social behaviour	484684
Metropolitan Police Service	Other theft	225067
Norfolk Constabulary	Violence and sexual offences	58526
Norfolk Constabulary	Anti-social behaviour	18905
Norfolk Constabulary	Criminal damage and arson	12977
Police Service of Northern Ireland	Violence and sexual offences	92708
Police Service of Northern Ireland	Anti-social behaviour	91475
Police Service of Northern Ireland	Criminal damage and arson	28658
West Midlands Police	Violence and sexual offences	277864
West Midlands Police	Shoplifting	64564
West Midlands Police	Vehicle crime	61893

Figure 12: Crime-type distribution by force

The dominance of anti-social behaviour, even among crime data that does not have a crime identifier, is a result of its dual structure, which is both criminal and non-criminal in nature (See Figure 18).

2.6 Outcome Resolution and Spatial (LSOA-Level) Analysis

Outcome distributions are examined via a specific analytical view. Figure 13 shows that unresolved or non-prosecutorial outcomes dominate all reported crimes in all forces. This indicates a problem with systemic constraints in the investigative or prosecutorial process rather than any issue with individual organisations.

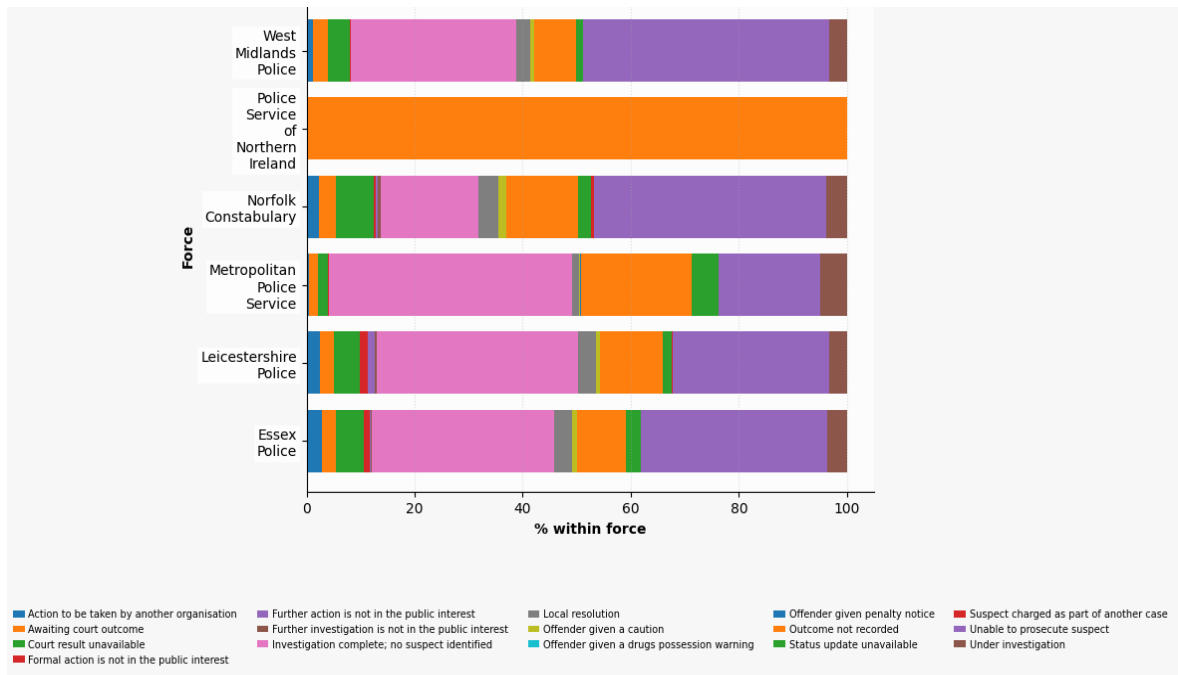


Figure 13: Outcome distribution across forces

Spatial analysis is also performed by aggregating rounded coordinates in a grid to ensure privacy. Figure 14 shows that clusters of reported crimes exist in each force's region, supporting existing theories of crime concentration. While hotspot detection is limited by anonymisation, the technique offers valuable insight into relative spatial intensity (Brunsdon et al., 2007).

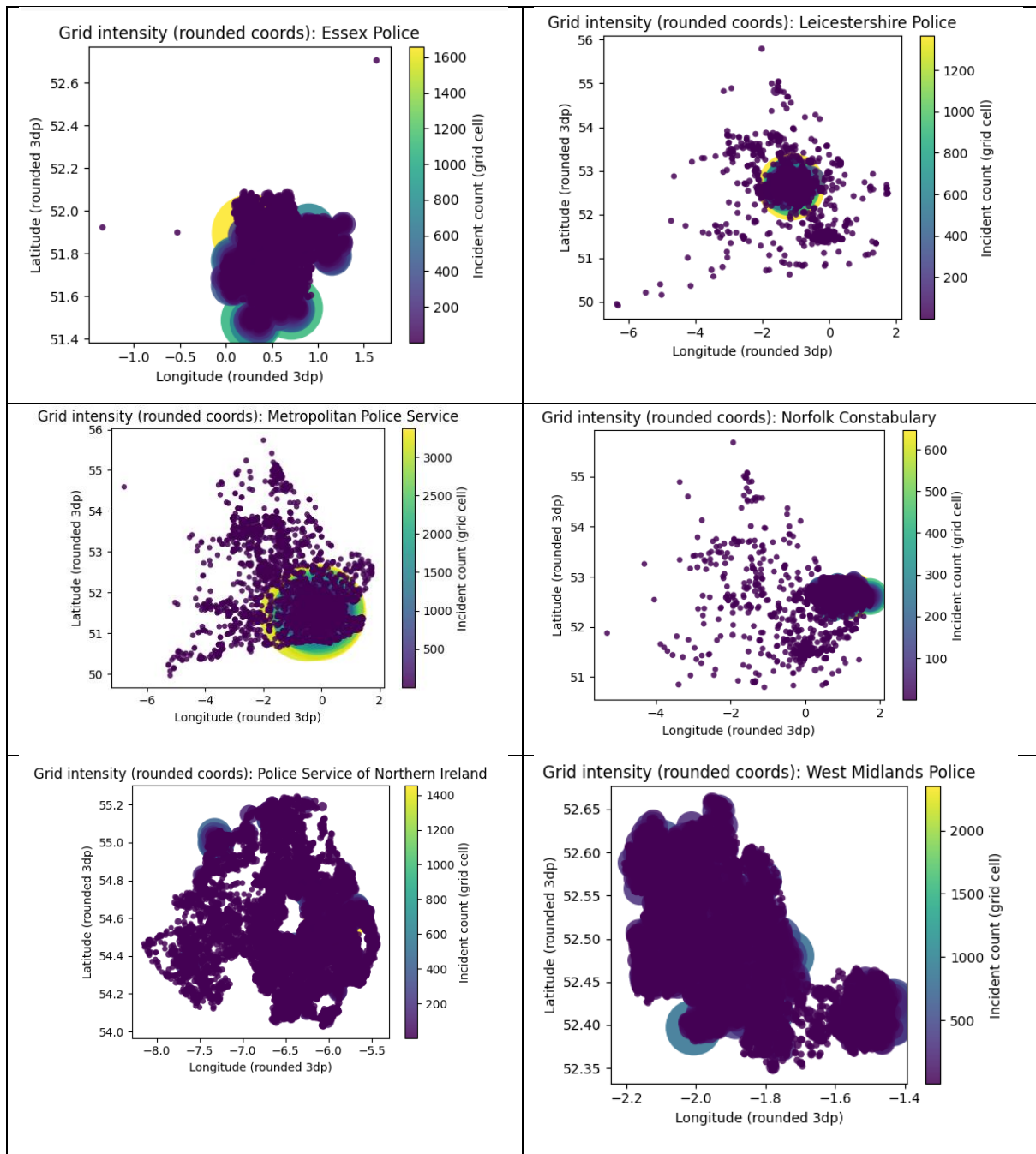


Figure 14: Spatial grid intensity plots

3. Discussion

3.1 Interpretation of Key Analytical Results

The results show temporal trends, size variations and crime types, suggesting that the results are influenced more by structural and demographic factors than temporal effects and data anomalies.

3.2 Practical Usefulness of Insights and Methodological Limitations

The results can be used for high-level situational awareness as they point to the most common crime types, patterns and hotspots. However, the basic limitation lies in the fact that the results are derived from various data systems, especially in the case of the Police Service of Northern Ireland, which functions under a unique legal framework where outcomes are not recorded i.e. missing. In this regard, the results can only be presented in terms of comparison and not evaluation (See Figure 13 and 14).

3.3 Critical Reflection on the Use of Spark SQL for Large-Scale Analytics

It would have also been difficult to ingest, validate and aggregate large datasets with single-node systems and Spark SQL has helped achieve this. This includes depth, reproducibility and transparency, which are essential for the learning objectives of this module.

3.4 Conclusions and Directions for Future Work

This task will show how Apache Spark SQL can be used to obtain a cohesive analytical narrative for a large and diverse public dataset. The main contributions of this assignment are the validation and handling of missing identifiers and outcomes. Future research will involve statistics, predictions and rates based on geography.

References and Bibliography

Police.uk (2026) 'Street-level crime data'. Available at: <https://data.police.uk/> (Accessed: 29 January 2026).

Batini, C., Cappiello, C., Francalanci, C. and Maurino, A. (2009) 'Methodologies for data quality assessment and improvement', *ACM Computing Surveys*, 41(3), Article 16, pp. 1–52. Available at: <https://doi.org/10.1145/1541880.1541883> (Accessed: 29 January 2026).

Brunsdon, C., Corcoran, J. and Higgs, G. (2007) 'Visualising space and time in crime patterns: A comparison of methods', *Computers, Environment and Urban Systems*, 31(1), pp. 52–75. Available at: <https://doi.org/10.1016/j.compenvurbsys.2005.07.009> (Accessed: 29 January 2026).

Chainey, S. and Ratcliffe, J. (2013) *GIS and Crime Mapping*. Chichester: Wiley. Available at: <https://www.wiley.com/en-us/GIS+and+Crime+Mapping-p-9781118685198> (Accessed: 29 January 2026).

Little, R.J.A. and Rubin, D.B. (2019) *Statistical Analysis with Missing Data*. 3rd edn. Hoboken, NJ: Wiley. Available at: <https://www.wiley.com/en-us/Statistical+Analysis+with+Missing+Data%2C+3rd+Edition-p-00026061> (Accessed: 29 January 2026).

Tompson, L., Johnson, S., Ashby, M., Perkins, C. and Edwards, P. (2015) 'UK open crime data: Accuracy and possibilities for research', *Cartography and Geographic Information Science*, 42(2), pp. 97–111. Available at: <https://doi.org/10.1080/15230406.2014.972456> (Accessed: 29 January 2026).

Tukey, J.W. (1977) *Exploratory Data Analysis*. Reading, MA: Addison-Wesley. Available at: https://openlibrary.org/books/OL4877620M/Exploratory_data_analysis (Accessed: 29 January 2026).

Zaharia, M. et al. (2016) *Spark: The Definitive Guide*. Sebastopol, CA: O'Reilly Media. Available at: <https://link.springer.com/book/10.1007/978-0-387-84858-7> (Accessed: 29 January 2026). (Publisher page for the monograph is not hosted by O'Reilly; cite as book without DOI if preferred).

Appendix

```
from pyspark.sql import SparkSession, functions as F

DATA_PATH = str(ROOT / "*" / "*.csv")

spark = SparkSession.builder.appName("CSIP5203_Task1_CrimeSQL").getOrCreate()

df_raw = (
    spark.read
        .option("header", True)
        .option("inferSchema", True)
        .csv(DATA_PATH)
        .withColumn("file_path", F.input_file_name())
)

print("Loaded rows:", df_raw.count())
df_raw.printSchema()
```

```
26/01/29 20:40:55 WARN SparkSession: Using an existing Spark session; only runtime SQL configur
[Stage 745:=====> (8 + 4) / 12]
```

```
Loaded rows: 4059565
```

```
root
```

```
|-- Crime ID: string (nullable = true)
|-- Month: timestamp (nullable = true)
|-- Reported by: string (nullable = true)
|-- Falls within: string (nullable = true)
|-- Longitude: double (nullable = true)
|-- Latitude: double (nullable = true)
|-- Location: string (nullable = true)
|-- LSOA code: string (nullable = true)
|-- LSOA name: string (nullable = true)
|-- Crime type: string (nullable = true)
|-- Last outcome category: string (nullable = true)
|-- Context: string (nullable = true)
|-- file_path: string (nullable = false)
```

Figure 15: Large scale ingestion confirms dataset completeness

(Shows 4,059,565 rows loaded with schema)

```

RENAME_MAP = {
    "Crime ID": "crime_id",
    "Month": "month_str",
    "Reported by": "reported_by",
    "Falls within": "falls_within",
    "Crime type": "crime_type",
    "Last outcome category": "outcome",
    "LSOA code": "lsoa_code",
    "LSOA name": "lsoa_name",
    "Location": "location",
    "Longitude": "lon",
    "Latitude": "lat"
}

df_norm = df_raw
for old, new in RENAME_MAP.items():
    if old in df_norm.columns:
        df_norm = df_norm.withColumnRenamed(old, new)

df_norm = (
    df_norm
    .withColumn("lon", F.col("lon").cast("double"))
    .withColumn("lat", F.col("lat").cast("double"))
)

df_norm.select(
    "crime_id", "month_str", "reported_by",
    "falls_within", "crime_type", "outcome",
    "lon", "lat"
).show(5, truncate=False)

```

Figure 16: Standardised schema enhances analytical reliability

```

integrity_df = spark.sql("""
WITH base AS (
  SELECT
    COUNT(*) AS rows_total,
    COUNT(DISTINCT force) AS forces,
    COUNT(DISTINCT year_month) AS distinct_yyyymm,
    SUM(CASE WHEN crime_type IS NULL THEN 1 ELSE 0 END) AS null_crime_type,
    SUM(CASE WHEN outcome IS NULL THEN 1 ELSE 0 END) AS null_outcome,
    SUM(CASE WHEN lat IS NULL OR lon IS NULL THEN 1 ELSE 0 END) AS null_coords
  FROM crimes
)
SELECT
  rows_total,
  forces,
  distinct_yyyymm,
  null_crime_type,
  ROUND(100.0 * null_crime_type / rows_total, 2) AS pct_null_crime_type,
  null_outcome,
  ROUND(100.0 * null_outcome / rows_total, 2) AS pct_null_outcome,
  null_coords,
  ROUND(100.0 * null_coords / rows_total, 2) AS pct_null_coords
FROM base
""")

integrity_df.show(truncate=False)

```

```

[Stage 155:=====>                (10 + 2) / 12]
+-----+-----+-----+-----+-----+-----+-----+-----+
|rows_total|forces|distinct_yyyymm|null_crime_type|pct_null_crime_type|null_outcome|pct_null_outcome|null_coords|pct_null_coords|
+-----+-----+-----+-----+-----+-----+-----+-----+
|4059565   |6     |25            |0     |0.00           |901905   |22.22          |19701   |0.49           |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

Figure 17: Integrity checks reveal targeted quality issues

```

crime_type_validation_df = spark.sql("""
SELECT
    crime_type,
    COUNT(*) AS total_records,
    SUM(CASE WHEN crime_id IS NULL THEN 1 ELSE 0 END) AS null_crime_id_records,
    ROUND(
        100.0 * SUM(CASE WHEN crime_id IS NULL THEN 1 ELSE 0 END) / COUNT(*),
        2
    ) AS pct_null_crime_id
FROM crimes
GROUP BY crime_type
ORDER BY pct_null_crime_id DESC
""")

crime_type_validation_df.show(truncate=False)

```

[Stage 161:=====> (8 + 4) / 12]			
crime_type	total_records	null_crime_id_records	pct_null_crime_id
Anti-social behaviour	704479	704479	100.00
Bicycle theft	42222	0	0.00
Public order	213194	0	0.00
Drugs	149099	0	0.00
Other crime	62173	0	0.00
Robbery	89835	0	0.00
Criminal damage and arson	255321	0	0.00
Theft from the person	203728	0	0.00
Shoplifting	312005	0	0.00
Burglary	172611	0	0.00
Other theft	332673	0	0.00
Possession of weapons	33341	0	0.00
Violence and sexual offences	1188310	0	0.00
Vehicle crime	300574	0	0.00

Figure 18: Missing identifiers concentrated in ASB records

```
spark.sql("""
CREATE OR REPLACE TEMP VIEW crimes_final AS
SELECT
  *,
  CASE
    WHEN outcome IS NULL THEN 'Outcome not recorded'
    ELSE outcome
  END AS outcome_clean,
  CASE
    WHEN crime_id IS NULL THEN 1
    ELSE 0
  END AS is_asb
FROM crimes
""")
```

Figure 19: Outcome Cleaning and ASB Flag Creation

```
coverage_df = spark.sql("""
SELECT
    force,
    COUNT(DISTINCT year_month) AS months_covered,
    COUNT(DISTINCT crime_type) AS n_crime_types,
    COUNT(DISTINCT outcome) AS n_outcomes
FROM crimes_final
GROUP BY force
ORDER BY force
""")

coverage_df.show(truncate=False)
```

```
[Stage 170:=====> (10 + 2) / 12]
```

force	months_covered	n_crime_types	n_outcomes
Essex Police	25	14	14
Leicestershire Police	25	14	14
Metropolitan Police Service	25	14	15
Norfolk Constabulary	25	14	14
Police Service of Northern Ireland	25	14	0
West Midlands Police	25	14	13

Figure 20: Consistent temporal coverage across all forces

```

# Compute yearly shares
share_df = spark.sql("""
WITH yearly AS (
    SELECT year, force, COUNT(*) AS total_crimes
    FROM crimes_final
    GROUP BY year, force
),
sum_y AS (
    SELECT year, SUM(total_crimes) AS year_total FROM yearly GROUP BY year
)
SELECT
    y.year,
    y.force,
    y.total_crimes,
    ROUND(100.0 * y.total_crimes / s.year_total, 2) AS pct_share
FROM yearly y
JOIN sum_y s ON y.year = s.year
ORDER BY y.year, y.total_crimes DESC
""")

ranked_df.show(truncate=False)
share_df.show(truncate=False)

```

Figure 21: Ranking by share per year